Laboratory work 1. ERD diagram.

1. Identify Entities:

Airports

Airlines

Flights

Passengers

Bookings

Booking Changes / History

Boarding Passes

Baggage

Baggage Checking

Security Checks

2. Determine Attributes:

1. Airports

- airport_id (INT, PK, AUTO_INCREMENT)
- airport_name (VARCHAR(100), NOT NULL)
- country (VARCHAR(50), NOT NULL)
- state (VARCHAR(50))
- city (VARCHAR(50), NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

2. Airlines

- airline_id (INT, PK, AUTO_INCREMENT)
- airline_code (VARCHAR(10), UNIQUE, NOT NULL)
- name (VARCHAR(100), NOT NULL)
- country (VARCHAR(50), NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

3. Flights

- flight_id (INT, PK, AUTO_INCREMENT)
- departing_gate (VARCHAR(10))
- arriving_gate (VARCHAR(10))
- airline_id (INT, FK -> Airlines.airline_id)
- departure_airport_id (INT, FK -> Airports.airport_id)
- arrival_airport_id (INT, FK -> Airports.airport_id)
- scheduled_departure_time (TIMESTAMP, NOT NULL)
- scheduled_arrival_time (TIMESTAMP, NOT NULL)
- actual_departure_time (TIMESTAMP)
- actual_arrival_time (TIMESTAMP)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

4. Passengers

- passenger_id (INT, PK, AUTO_INCREMENT)
- first_name (VARCHAR(50), NOT NULL)
- last_name (VARCHAR(50), NOT NULL)
- gender (VARCHAR(10))
- date_of_birth (DATE, NOT NULL)
- country_of_citizenship (VARCHAR(50), NOT NULL)
- country_of_residence (VARCHAR(50), NOT NULL)
- passport_number (VARCHAR(20), UNIQUE, NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

5. Bookings

- booking_id (INT, PK, AUTO_INCREMENT)
- flight_id (INT, FK -> Flights.flight_id)
- passenger_id (INT, FK -> Passengers.passenger_id)
- status (VARCHAR(20), NOT NULL)
- booking_platform (VARCHAR(50))
- ticket_price (DECIMAL(10,2), NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

6. Booking_Changes

- change_id (INT, PK, AUTO_INCREMENT)
- booking_id (INT, FK -> Bookings.booking_id)
- change_description (TEXT)
- changed_at (TIMESTAMP)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

7. Boarding_Passes

- boarding_pass_id (INT, PK, AUTO_INCREMENT)
- booking_id (INT, FK -> Bookings.booking_id)
- seat (VARCHAR(10), NOT NULL)
- boarding_time (TIMESTAMP, NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

8. Baggage

- baggage_id (INT, PK, AUTO_INCREMENT)
- booking_id (INT, FK -> Bookings.booking_id)
- weight_kg (DECIMAL(5,2), NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

9. Baggage_Checking

- baggage_checking_id (INT, PK, AUTO_INCREMENT)
- booking_id (INT, FK -> Bookings.booking_id)
- passenger_id (INT, FK -> Passengers.passenger_id)
- check_results (VARCHAR(100), NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

10. Security_Checks

- security_check_id (INT, PK, AUTO_INCREMENT)
- passenger_id (INT, FK -> Passengers.passenger_id)
- check_results (VARCHAR(100), NOT NULL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

3.Identify Relationships:

1. Airlines -> Flights

- Type: One-to-Many (1:N)
- Description: One airline can operate multiple flights. Each flight belongs to exactly one airline.

2. Airports -> Flights (Departure)

- Type: One-to-Many (1:N)
- Description: One airport can be the departure location for multiple flights. Each flight has one departure airport.

3. Airports -> Flights (Arrival)

- Type: One-to-Many (1:N)
- Description: One airport can be the arrival location for multiple flights. Each flight has one arrival airport.

4. Passengers -> Bookings

- Type: One-to-Many (1:N)
- Description: One passenger can make multiple bookings. Each booking belongs to one passenger.

5. Flights -> Bookings

- Type: One-to-Many (1:N)
- Description: One flight can have multiple passenger bookings. Each booking is for a single flight.

6. Bookings -> Booking_Changes

- Type: One-to-Many (1:N)
- Description: One booking can have multiple recorded change entries over time.

7. Bookings -> Boarding_Passes

- Type: One-to-One (1:1)
- Description: Each booking corresponds to exactly one issued boarding pass.

8. Bookings -> Baggage

- Type: One-to-Many (1:N)
- Description: One booking can have multiple registered pieces of baggage.

9. Bookings -> Baggage_Checking

- Type: One-to-Many (1:N)
- Description: One booking can have multiple baggage checking records.

10. Passengers -> Security_Checks

- Type: One-to-Many (1:N)
- Description: One passenger can undergo multiple security checks over time.

4.Draw Entity Boxes:

Independent rectangular blocks were created in the diagram editor for each of the 10 identified entities: Airports, Airlines, Flights, Passengers, Bookings, Booking_Changes, Boarding_Passes,

Baggage, Baggage_Checking, and Security_Checks. Each box represents a core system component within the international airport database structure.

5.Add Attributes:

Attributes and suitable data types were defined inside each entity box based on system requirements. Primary Keys (PK) with auto-increment constraints were set for each table, Foreign Keys (FK) were marked to enable logical references, and required properties like NOT NULL and UNIQUE constraints were applied to maintain data integrity.

6.Connect Entities with Relationships

Entities were linked using relational connectors based on business domain rules. One-to-Many (1:N) relationships were established between parent entities and child tables (such as Airlines to Flights, Passengers to Bookings, and Bookings to Baggage), while a One-to-One (1:1) connection was defined between Bookings and Boarding_Passes

7 .Document the Diagram:

Notation & Symbol Legend:

- **PK (Primary Key):** Unique attribute used to identify each record in a table.
- **FK (Foreign Key):** Attribute that creates a relationship and links to the primary key of another table.
- **1:N (One-to-Many):** Relationship where one parent record connects to multiple child records.
- **1:1 (One-to-One):** Relationship where one record links to exactly one corresponding record.

